



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



Practical 5: Doubly Linked List and Circular Linked List Operations

Practical Aim

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation. Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

Problem 1

For Code:

https://github.com/patelaarnav882-collab/FDSA_A2_SHP

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\DSA_clg> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5\" ; if ($?) { g++ pr5_p1.cpp -o pr5_p1 } ; if ($?) { .\pr5_p1 }

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 1
Enter song name: asdfg
Song added at beginning.
Playlist: asdfg

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 1
Enter song name: lkjh
Song added at beginning.
Playlist: lkjh asdfg

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 2
Enter song name: qwert
Song added at end.
Playlist: lkjh asdfg qwert
```

```

PS C:\Users\ADMIN\OneDrive\Desktop\DSA_clg> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5\" ; if ($?) { g++ pr5_p1.cpp -o pr5_p1 } ; if ($?) { .\pr5_p1 }

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 3
Enter existing song name: qwert
Enter new song name: poiuy
Song inserted after qwert.
Playlist: lkjh asdfg qwert poiuy

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 6
Playlist: lkjh asdfg qwert poiuy

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 5
Total Songs = 4
Playlist: lkjh asdfg qwert poiuy

```

```

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 4
First song removed.
Playlist: asdfg qwert poiuy

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 6
Playlist: asdfg qwert poiuy

===== MUSIC PLAYLIST MENU =====
1. Add Song at Beginning
2. Add Song at End
3. Insert Song After Given Song
4. Remove First Song
5. Count Songs
6. Display Playlist
7. Exit
Enter choice: 7
Exiting...
PS C:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5>

```

Problem Faced:

Problem:

Firstly forgot to add previous pointer.

Solution:

Revised from notes

Practical Aim

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation. Describe how your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

Problem 2

For Code:

https://github.com/patelaarnav882-collab/FDSA_A2_SHP

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5\" ; if ($?) { g++ pr5_p2.cpp -o pr5_p2 } ; if ($?) { .\pr5_p2 }

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 1
Enter student name: A
A joined the circle.
Students: A

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 1
Enter student name: B
B joined the circle.
Students: A B

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 1
Enter student name: C
C joined the circle.
Students: A B C
```

```

PS C:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5\" ; if ($?) { g++ pr5_p2.cpp -o pr5_p2 } ; if ($?) { .\pr5_p2 }

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 1
Enter student name: D
D joined the circle.
Students: A B C D

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 2
Enter existing student: B
Enter new student: E
E joined after B.
Students: A B E C D

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 4
Students: A B E C D

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 3
Enter student to leave: D
D left the circle.

===== MENU =====
1. Student Join (End)
2. Student Join After
3. Student Leave
4. Display Circle
5. Exit
Enter choice: 5
Exiting...
PS C:\Users\ADMIN\OneDrive\Desktop\DSA_clg\practical 5>

```

Problem Faced:

Problem:

Forgot to connect tail with head

Solution:

Revised from notes

Outcome/Conclusion:

Students will be able to implement doubly and circular linked list operations, articulate how maintaining two links or a circular connection changes insertion, deletion, and traversal logic, and handle edge cases such as broken circles, missing nodes, and repeated values in dynamic sequences.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

Ans:- In a doubly linked list, after deleting the front node, you must update the new first node's **prev pointer** to NULL. If you forget to do this, the prev pointer will still point to the deleted node, creating a dangling pointer. This can lead to incorrect traversal, undefined behavior, or program crashes. In a **singly linked list**, this responsibility does not exist because nodes only have a next pointer.

2. [Problem 2] If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

Ans:- In a circular linked list, traversal should stop when you return to the starting (head) node, indicating that all nodes have been visited once. If the last node's link is set to NULL, the list is no longer circular. The traversal can no longer return to the head, breaking the circular property and making the stopping condition invalid.